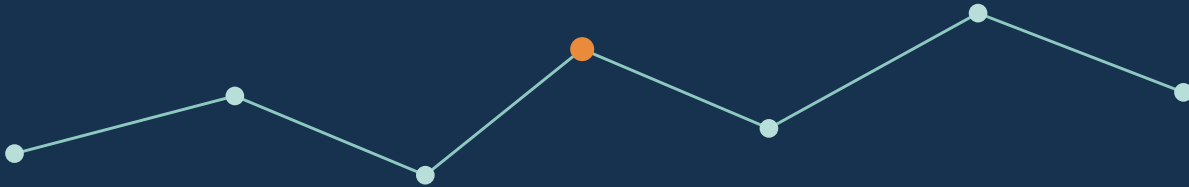


RUNOPSY

AI agent alıřtırmaları iin yerel-ncelikli, aık kaynak, nedensel hata analizi, etki yayılımı ve kontroll replay zellikli bağımsız CLI rn

Proje sahibi: Vahit Feryad
Belge tr: rn + arařtırma mimarisi
Karar: Bağımsız rn + ilk adaptr olarak Hermes
Srm: 0.2 - karar verici srm



Belge Kontrolü

Alan	Değer
Belge adı	Runopsy - AI Agent Failure Intelligence Platformu Proje Tasarım Belgesi
Ürün çalışma adı	Runopsy
Durum	Karar verici anlatımı + teknik ürün mimarisi
Sürüm	0.2
Tarih	30 Temmuz 2026
Proje sahibi	Vahit Feryad
Hedef okuyucu	Müşteri, yatırımcı, ürün/teknoloji yöneticisi, geliştirici ve araştırmacı
Belge amacı	Ürünün ne olduğunu, nasıl kullanıldığını, neden değer ürettiğini ve nasıl geliştirileceğini tek belgede netleştirmek
Ana karar	Runopsy çekirdeğini runtime bağımsız kurmak; Hermes'i ilk adaptör olarak fork etmeden desteklemek
Lisans yaklaşımı	Runopsy çekirdeği için Apache-2.0 önerisi; Hermes MIT bildirimleri korunur; yayımdan önce bağımlılık lisans taraması yapılır
Pazar adı	'Runopsy' ürün çalışma adıdır. Hızlı web taramasında belirgin çakışma görülmemiştir; marka, paket ve alan adı uygunluğu hukuki araştırmayla kesinleştirilmelidir.

Belgenin karar özeti

Runopsy başka bir genel amaçlı agent değildir. Farklı agent runtime'larına bağlanan bağımsız bir teşhis ve kurtarma ürünüdür. İlk adaptör Hermes'in görev yürütme, provider, tool, session ve checkpoint altyapısını kullanır. Ürün değeri 3B görselleştirme değil; kanıtlı teşhis, nedensel yayılım analizi ve doğrulanabilir kurtarma başarısıdır.

A. Runopsy Nedir? - 60 Saniyelik Ürün Tanımı

Runopsy, yazılım geliştirme görevlerini terminalden çalıştıran ve agent başarısız olduğunda yalnızca log göstermek yerine hatanın olası başlangıcını, kanıtlarını, sonraki adımlara etkisini ve güvenli yeniden çalıştırma seçeneğini sunan reliability-first coding agent CLI ürünüdür.

Soru	Net cevap
Ürün kategorisi nedir?	OpenCode/Hermes benzeri coding agent CLI + yerleşik failure intelligence ve replay
Kullanıcı ne yaptırır?	Kod yazma, hata düzeltme, test, refactor, repository inceleme, terminal/Git ve sınırlı otomasyon görevleri
Asıl farkı nedir?	Agent run bozulduğunda ilk şüpheli adımı, kanıtı, yayılımı ve doğrulama/replay planını aynı terminalde sunması
Hermes ürünün kendisi mi?	Hayır. Hermes ilk runtime adaptörü ve ilk kullanıcı edinme kanalıdır
Bulut zorunlu mu?	Hayır. Trace, teşhis ve veri katmanı varsayılan olarak lokaldir
Ücretli model zorunlu mu?	Hayır. Lokal model, OpenRouter BYOK veya tamamen tokensız deterministik mod kullanılabilir
3B görünüm ürün mü?	Hayır. Opsiyonel bir inceleme arayüzüdür; ürün değeri kanıtlı teşhis ve replay'dir



Runopsy, coding agent görevini yürütürken gözlem, teşhis ve kontrollü kurtarmayı aynı terminal akışında birleştirir.

Tek cümlelik satış anlatımı

Runopsy, coding agent'inize görev yaptırırken başarısız koşulların nerede bozulduğunu kanıtlarıyla gösterir ve kullanıcı onayıyla doğru noktadan yeniden denemenizi sağlar.

B. Kullanıcılar Terminali Hangi Amaçlarla Kullanır?

Birincil kullanım coding'dir. Ancak terminal yalnızca kod üreten bir sohbet kutusu değildir; görev yürütme, araç kullanma, doğrulama, teşhis ve kurtarma yüzeyidir.

B.1 Kullanıcı Roller

Rol	Terminalde yaptığı iş	Runopsy'den beklediği değer
Yazılım geliştirici	Feature, bug fix, refactor, test ve repository incelemesi	Hatalı run'ı hızlı açıklama; güvenli replay
AI/agent mühendisi	Prompt, model, tool, memory ve multi-agent akışları	Failure onset, handoff ve propagation analizi
Araştırmacı/eval mühendisi	Agent deneyi, fault injection, model/policy karşılaştırması	Tekrarlanabilir trace, etiket ve benchmark
Platform/DevOps ekibi	Runtime, provider, policy ve maliyet yönetimi	Run sağlığı, bütçe ve regresyon görünürlüğü
Teknik lider	Risk, teslim süresi ve kalite kararı	Ölçülebilir teşhis süresi ve recovery başarısı
Güvenlik/uyum	Riskli tool eylemleri ve audit incelemesi	Policy gate, redaction, kanıt zinciri ve onay kaydı

B.2 Terminaldeki Beş İş Grubu

İş grubu	Örnek	Runopsy rolü
Üret	Kod, test, config, migration veya dokümantasyon oluştur	Agent runtime'ını başlatır ve koşuyu kaydeder
Değiştir	Bug fix, refactor, dependency veya performans iyileştirmesi	Dosya/state değişimlerini checkpoint ile ilişkilendirir
Doğrula	Test, lint, type-check, build ve evaluator çalıştır	Başarı kriterini ve ilk gözlenen hatayı kaydeder
Teşhis et	Neden test bozuldu, tool yanlış çağrıldı veya agent döngüye girdi?	Aday başlangıç, kanıt, güven ve etkilenen adımları sunar
Kurtar	Checkpoint'e dön, session fork et veya farklı model/policy ile dene	Planı dry-run gösterir; onay sonrası replay başlatır

Ürün sınırı

Runopsy genel amaçlı kişisel asistan değildir. İlk pazarı; repository ve terminal araçlarıyla çalışan coding/engineering agent kullanıcılarıdır.

C. Somut Kullanım Senaryosu - Coding Agent Hatası

Görev: Kullanıcı, ödeme servisindeki başarısız entegrasyon testini düzeltmesini ister. Agent repository'yi inceler, iki dosyayı değiştirir ve testleri çalıştırır. Bir konfigürasyon varsayımı yanlış olduğu için ilk hata 9. adımda oluşur; test hatası ise ancak 14. adımda görünür.

```
# Görevi çalıştır
runopsy run --adapter hermes --provider openrouter -- "Ödeme servisindeki başarısız entegrasyon testini düzelt"

# Son koşuyu tokensız yapısal olarak teşhis et
runopsy diagnose latest --mode deterministic

# 9. adımın kanıtını ve etki zincirini incele
runopsy evidence latest --step 9

# Kullanıcı onayından önce replay planını göster
runopsy replay latest --from-step 9 --mode fork --dry-run
```

C.1 Kullanıcının Gördüğü Teşhis

Çıktı	Örnek kullanıcı cevabı
Gözlenen hata	Adım 14: integration test exit code 1
Kök neden adayı	Adım 9: test ortamı için yanlış endpoint/config varsayımı
Kanıt	Config diff, tool output, ilk invariant ihlali ve test log satırı
Yayılım	Adım 9 -> dosya değişimi -> servis başlatma -> adım 14 test hatası
Güven	Orta; replay yapılmadığı için kesin kök neden olarak sunulmaz
Önerilen eylem	Adım 8 checkpoint'inden fork; doğru config ile yalnız etkilenen testleri çalıştır

C.2 Runopsy Olmadan / Runopsy ile

Runopsy olmadan	Runopsy ile
Kullanıcı uzun trace ve test loglarını manuel okur	İlk şüpheli adım ve ilişkili evidence öne çıkar
Son hata gerçek başlangıç sanılabilir	Observed failure ve suspected onset ayrı gösterilir
Görev baştan ve pahalı biçimde tekrar edilir	İlgili checkpoint/fork noktasından kontrollü replay planlanır
Neden düzeldiği bilinmeden yeni çıktı kabul edilir	Original ve replay run karşılaştırılarak doğrulama kaydı üretilir

D. Ürün Nasıl Çalışır, Ne Zaman Token Harcar?

Aşama	Ne yapar?	Ek LLM tokenı	Varsayılan
Agent görevi	Kod üretir, tool çağırır, dosya değiştirir ve test çalıştırır	Evet	Seçilen görev modeli
Trace capture	LLM/tool/session event'lerini yerel kaydeder	Hayır	Her zaman açık
Deterministik analiz	Exception, loop, state diff, dependency ve propagation inceler	Hayır	Her zaman açık
Semantik analiz	Yalnız şüpheli span'da niyet, claim-evidence ve handoff kontrolü yapar	Opsiyonel	Bütçe/policy ile
Replay	Seçili noktadan modeli ve gerekli tool'ları yeniden çalıştırır	Evet	Kullanıcı onaylı
2B/3B görünüm	Kayıtlı trace ve graph verisini görselleştirir	Hayır	2B varsayılan

D.1 Kurulum ve Dağıtım Seçenekleri

Mod	Model	Veri	Hedef kullanıcı
Tam lokal ve ücretsiz	Ollama/llama.cpp	Cihazdan çıkmaz	Bireysel geliştirici, hassas repository
OpenRouter BYOK	Free router veya seçilen model	Trace lokal; seçili prompt/provider'a gider	Kolay başlangıç ve model esnekliği
Rules-only	Model yok	Tam lokal	Tokensız teşhis, CI ve güvenlik politikaları
Self-hosted takım	Lokal/BYOK	Kurum altyapısı	Paylaşılan run geçmişi ve ekip politikaları
Managed cloud	Gelecek seçenek	Açık opt-in	Kurumsal ölçek; MVP kapsamında değil

Kritik maliyet cevabı

Runopsy'nin kayıt ve yapısal teşhis katmanı token harcamaz. Token yalnızca normal agent görevi, kullanıcının açtığı semantik analiz ve replay sırasında kullanılır.

E. Müşteriye Sağlanan Değer ve Nasıl Ölçüleceği

Müşteri problemi	Runopsy yeteneği	Ölçülecek KPI	Kanıt yöntemi
Başarısız run'ı anlamak uzun sürüyor	Aday onset + evidence	Median time-to-diagnosis	Manuel inceleme ile A/B pilot
Son hata ile gerçek başlangıç karışıyor	Propagation graph	Top-1/top-3 onset recall	Etiketli fault injection
Her hata için görev baştan çalışıyor	Checkpoint/fork replay	Recovery rate ve rerun cost	Original/replay pair
Agent gereksiz token tüketiyor	Loop/retry/budget detector	Ek token ve durdurulan loop	Provider usage + trace
Yanlış teşhise güven oluşuyor	Calibrated status + evidence	False-cause rate / calibration	Human label + replay
Riskli tool eylemleri belirsiz	Policy gate + approval	Blocked unsafe action / false block	Kontrollü test suite

E.1 ROI Nasıl Hesaplanır?

Aylık net değer = (başarısız run sayısı x run başına kazanılan teşhis süresi x yüklü mühendislik maliyeti) + önlenen gereksiz token/replay maliyeti + önlenen riskli işlem maliyeti - Runopsy toplam sahip olma maliyeti.

Belgede parasal kazanç iddiası verilmez. Bu değişkenler 4-6 haftalık pilotta müşterinin gerçek iş yükünden ölçülmeden ROI beyanı yapılmamalıdır.

E.2 Kullanıcı, Alıcı ve Onaylayıcı

Rol	Karar sorusu
Günlük kullanıcı	Agent hatasını daha hızlı anlayıp güvenle devam edebiliyor muyum?
Ekonomik alıcı: Engineering/AI Platform Lead	Teslim süresi, token maliyeti ve incident yükü azalıyor mu?
Teknik onaylayıcı	Runtime/framework bağımsız mı; kurulumu ve entegrasyonu yönetilebilir mi?
Güvenlik/uyum onaylayıcısı	Kod ve trace nerede tutuluyor; replay ve tool yetkileri kontrol ediliyor mu?

F. Pazar Konumu, Farklılaşma ve İş Modeli

Kategori	Güçlü olduğu alan	Runopsy'nin konumu
Coding agent CLI'ları	Kod üretme, tool kullanma ve repository görevi yürütme	Aynı terminal deneyimine failure localization, evidence ve replay ekler
LLM/agent observability	Trace, latency, token, eval ve dashboard	Dashboard'dan önce terminal-native teşhis ve kontrollü recovery'ye odaklanır
APM/root-cause araçları	Servis log/metric/trace korelasyonu	Agent'e özgü prompt, tool, state, memory ve handoff semantiğini işler
Agent evaluation framework'leri	Dataset, scorer, regression ve sandbox	Canlı run teşhisi ile benchmark/replay kanıtını aynı veri modelinde birleştirir

Savunulabilir fark

Moat, yalnız görselleştirme veya LLM ile log özetleme değildir. Etiketli failure corpus, frameworkler arası normalize trace graph, kalibre onset/propagation ölçümü ve replay ile doğrulanmış recovery verisidir.

F.1 Önerilen Open-Core Model

Ücretsiz/açık kaynak çekirdek	Daha sonra ticari takım ürünü
Runopsy CLI, lokal collector ve deterministik dedektörler	Paylaşılan dashboard, ekip run geçmişi ve karşılaştırma
Hermes adaptörü, adapter SDK ve community adaptörleri	RBAC, SSO, audit export ve merkezi policy yönetimi
Lokal 2B görünüm, evidence raporu ve basic replay	Alerting, trend/regression analizi ve uzun dönem retention
Runopsy-Bench şeması ve temel fault-injection örnekleri	Kurumsal connector, SLA, özel destek ve self-hosted paket

F.2 Pazara Giriş Sırası

- Önce bireysel coding-agent geliştiricileri: tek komut kurulum, lokal demo, açık kaynak.
- Sonra agent framework ekipleri: adapter SDK, benchmark ve failure corpus katkıları.
- Ardından AI platform ekipleri: self-hosted paylaşım, policy ve regresyon görünümü.
- Kurumsal özellikler ancak gerçek pilotlarda tekrar eden ödeme isteği doğrulandıktan sonra geliştirilir.

G. MVP Kararı, Ürün Sınırı ve Sık Sorulan Sorular

G.1 İlk Sürümde Kullanıcıya Verilecekler

MVP içinde	MVP dışında
Bağımsız Runopsy CLI ve lokal çalışma	Genel amaçlı sohbet/personal assistant
Coding görevleri için Hermes runtime adaptörü	İlk günde tüm agent framework'leri
Trace/state capture ve deterministik dedektörler	Her koşulda kesin kök neden garantisi
Aday onset, evidence ve propagation görünümü	Otonom ve onaysız repo değişikliği/recovery
Checkpoint/session fork tabanlı onaylı replay	3B görünümün varsayılan veya ana ürün olması
Lokal model, OpenRouter BYOK ve rules-only mod	Zorunlu bulut hesabı veya ücretli model bağımlılığı
Runopsy-Bench ve fault-injected demo seti	MVP'de kurumsal SaaS platformu

G.2 Karar Verici SSS

Soru	Cevap
Bu bir coding terminali mi?	Evet. Birincil işlevi coding/repository görevleri yürüten agent CLI olmaktadır
Hermes mi satıyoruz?	Hayır. Runopsy bağımsız üründür; Hermes ilk ücretsiz runtime adaptörüdür
Hata analizi token harcar mı?	Deterministik analiz hayır; semantik analiz ve replay açılırsa evet
Ücretli model gerekli mi?	Hayır. Lokal model veya rules-only mod mümkündür; OpenRouter isteğe bağlıdır
Runopsy kesin kök neden bulur mu?	Replay/insan doğrulaması yoksa yalnız kanıtlı ve sıralı aday sunar
Kod cihazdan çıkar mı?	Varsayılan hayır; provider'a gönderilecek içerik ve redaction policy kullanıcı kontrolündedir
Runopsy kendi başına dosya değiştirir mi?	Görev runtime'ı değiştirir; Runopsy kaydeder, policy uygular ve riskli replay için onay ister
3B neden var?	Büyük multi-agent graph'larda opsiyonel keşif içindir; 2B timeline varsayılandır
Müşteri neden ödeme yapar?	Takım paylaşımı, merkezi policy, audit, retention, trend/regression ve kurumsal destek için
Başarı nasıl kanıtlanır?	Etiketli failure seti, gerçek opt-in pilot ve original/replay karşılaştırmasıyla

Go / no-go kararı

MVP, üç şeyi kanıtlamalıdır: geliştirici teşhis süresini azaltmak, ilk hata başlangıcını baseline'lardan daha doğru sıralamak ve güvenli replay ile anlamlı recovery üretmek. Bu üç sonuç ölçülmeden 3B arayüz veya geniş kurumsal platform yatırımı yapılmamalıdır.

Kısaltmalar ve Terimler

Kısaltma	İngilizce açılım	Türkçe karşılık / kullanım
Runopsy	Run + Autopsy	Ürün çalışma adı; agent run incelemesi ve hata otopsis
CFA	Causal Failure Analysis	Nedensel Hata Analizi; Runopsy çekirdeğinin analiz yöntemi
RCA	Root Cause Analysis	Kök Neden Analizi
AI	Artificial Intelligence	Yapay Zeka
LLM	Large Language Model	Büyük Dil Modeli
CLI	Command-Line Interface	Komut Satırı Arayüzü
TUI	Terminal User Interface	Terminal Kullanıcı Arayüzü
UI / UX	User Interface / User Experience	Kullanıcı Arayüzü / Kullanıcı Deneyimi
API	Application Programming Interface	Uygulama Programlama Arayüzü
SDK	Software Development Kit	Yazılım Geliştirme Kiti
ACP	Agent Client Protocol	Agent istemcisi ile runtime arasındaki iletişim protokolü
MCP	Model Context Protocol	Model ile araç/veri kaynaklarını bağlayan protokol
OTel	OpenTelemetry	Vendor-bağımsız telemetry standardı
OTLP	OpenTelemetry Protocol	OpenTelemetry veri taşıma protokolü
BYOK	Bring Your Own Key	Kullanıcının kendi provider API anahtarını kullanması
MVP	Minimum Viable Product	Minimum Uygulanabilir Ürün
DAG	Directed Acyclic Graph	Yönlü Döngüsüz Graf
PII	Personally Identifiable Information	Kişisel Olarak Tanımlanabilir Bilgi
SBOM	Software Bill of Materials	Yazılım Bileşen Envanteri
JSONL	JSON Lines	Her satırda bir JSON nesnesi bulunan kayıt biçimi
SLA	Service Level Agreement	Hizmet Seviyesi Anlaşması / hedefi
CI	Continuous Integration	Sürekli Entegrasyon
CPU / GPU	Central / Graphics Processing Unit	Merkezi / Grafik İşlem Birimi
RAM / VRAM	Random Access / Video Random Access Memory	Sistem / ekran kartı belleği
2B / 3B	Two-Dimensional / Three-Dimensional	İki Boyutlu / Üç Boyutlu görselleştirme

İçindekiler

A. Runopsy Nedir? - 60 Saniyelik Ürün Tanımı	3
B. Kullanıcılar Terminali Hangi Amaçlarla Kullanır?	4
B.1 Kullanıcı Rollerini	4
B.2 Terminaldeki Beş İş Grubu	4
C. Somut Kullanım Senaryosu - Coding Agent Hatası	5
C.1 Kullanıcının Gördüğü Teşhis	5
C.2 Runopsy Olmadan / Runopsy ile	5
D. Ürün Nasıl Çalışır, Ne Zaman Token Harcar?	6
D.1 Kurulum ve Dağıtım Seçenekleri	6
E. Müşteriye Sağlanan Değer ve Nasıl Ölçüleceği	7
E.1 ROI Nasıl Hesaplanır?	7
E.2 Kullanıcı, Alıcı ve Onaylayıcı	7
F. Pazar Konumu, Farklılaşma ve İş Modeli	8
F.1 Önerilen Open-Core Model	8
F.2 Pazara Giriş Sırası	8
G. MVP Kararı, Ürün Sınırı ve Sık Sorulan Sorular	9
G.1 İlk Sürümde Kullanıcıya Verilecekler	9
G.2 Karar Verici SSS	9
1. Yönetici Özeti	15
2. Problem Tanımı ve Fırsat	15
2.1 Problem	15
2.2 Pazar Boşluğu	16
3. Proje Amacı, Hedefler ve Kapsam	16
3.1 Ana Amaç	16
3.2 Ölçülebilir Hedefler	16

3.3 İlk Sürümde Kapsam Dışı	16
4. Ürün Konumlandırması ve Kullanıcılar	18
4.1 Hedef Kullanıcılar	18
4.2 Temel Kullanım Senaryoları	18
4.3 Ürün Vaadi ve Rakiplerden Ayrışma	18
5. Önerilen Mimari	19
5.1 Mimari İlkeler	19
5.2 Bileşenler	20
6. İlk Adaptör: Hermes Entegrasyon Tasarımı	20
6.1 Plugin Paket Yapısı	21
6.2 Plugin Sınırları ve Upstream Stratejisi	21
7. Trace ve Veri Modeli	22
7.1 Ana Düğüm Türleri	22
7.2 Ana Kenar Türleri	22
7.3 Örnek Normalize Event	23
7.4 Saklama	23
8. Runopsy Causal Failure Analysis Engine	23
8.1 Analiz Katmanları	24
8.2 Aday Skorlama	24
8.3 Teşhis Statüleri	24
9. Hata Taksonomisi	25
10. Replay ve Kurtarma Tasarımı	26
10.1 Replay Seviyeleri	26
10.2 Replay Güvenlik Kontrolleri	26
10.3 Counterfactual Doğrulama	26
11. Model ve Provider Stratejisi	26
11.1 Kullanıcı Kurulumu	27

11.2 Actor ve Diagnostician Ayrımı	27
12. Token ve Maliyet Kontrolü	27
12.1 Bütçe Modları	28
12.2 Veri Minimizasyonu	28
13. Teknoloji Yığını ve Kütüphaneler	29
13.1 Backend ve CLI	29
13.2 Model ve Evaluation	29
13.3 Web ve Görselleştirme	29
13.4 Kalite, Paketleme ve Dağıtım	30
14. Companion CLI ve Kullanıcı Deneyimi	30
14.1 Hermes İçindeki Runopsy Slash Komutları	31
14.2 Lokal Model Onboarding	31
15. 2B ve 3B Görselleştirme Tasarımı	31
15.1 Varsayılan 2B Görünüm	31
15.2 Opsiyonel 3B Failure Map	31
16. Güvenlik, Gizlilik ve Yönetişim	32
16.1 Varsayılan Güvenli Mod	32
17. Runopsy-Bench: Bilimsel Doğrulama Planı	32
17.1 Veri Katmanları	33
17.2 Metrikler	33
17.3 Karşılaştırılacak Baseline'lar	33
18. Test Stratejisi ve Kalite Kapıları	34
18.1 MVP Kabul Kriterleri	34
19. Repository ve API Tasarımı	35
19.1 Monorepo Önerisi	35
19.2 Lokal API	35
20. Yol Haritası ve Karar Kapıları	36

20.1 Öncelik Sırası	36
21. Risk Kaydı	36
22. Açık Kaynak ve Piyasaya Çıkış Stratejisi	37
22.1 Lisans ve Ürün Modeli	37
22.2 İlk Dağıtım Kanalları	37
22.3 Görünürlük İçin Kanıt Paketi	37
23. Uygulama Kararları	38
24. İlk Sprint İçin Somut İş Listesi	38
25. Kaynaklar	39

1. Yönetici Özeti

Runopsy, farklı yapay zeka agent runtime'larında çalışan görevlerin nerede, neden ve nasıl bozulduğunu belirleyen bağımsız bir üründür. Causal Failure Analysis (CFA - Nedensel Hata Analizi) çekirdeği olası hata başlangıcını ve downstream etkilerini gösterir; uygun durumda checkpoint veya session fork üzerinden kontrollü yeniden çalıştırma yapar. Hermes yalnızca ilk runtime adaptörü ve ilk dağıtım yüzeyidir.

Pazar konumlandırması: Görev yapan ve başarısızlığını açıklayıp kırılma noktasından yeniden deneyebilen agent CLI. OpenCode gibi ürünler görev icrasını, Hermes hafıza ve öğrenme döngüsünü, LangSmith ve Langfuse ise observability'yi öne çıkarır. Bu projenin iddiası; observability, failure localization ve replay'i agent yürütme akışının içinde birleştirmektir.

Karar alanı	Seçim	Gerekçe
Agent runtime	Runopsy + Hermes adaptörü	Hazır CLI/TUI, model provider'ları, tool loop, hafıza, session ve checkpoint özellikleri
Entegrasyon	Plugin + companion engine	Upstream fork maliyetini azaltır; Runopsy motorunun framework bağımsız kalmasını sağlar
Çalışma modeli	Local-first, BYOK	Kullanıcının kodu ve trace'i lokal kalabilir; ortak API anahtarı dağıtılmaz
Analiz	Deterministik-first, LLM-on-demand	Her trace için token yakılmaz; yalnızca semantik belirsizlikte model çağrılır
Arayüz	TUI + lokal 2B, opsiyonel 3B	Kısa trace'lerde 2B okunabilirlik; multi-agent ve uzun-horizon çalışmalarda 3B topoloji
Doğrulama	Runopsy-Bench + counterfactual replay	Kök neden iddiası görsel sezgiye değil, etiketli veri ve müdahale testine dayanır

Tek cümlelik ürün vaadi

An open-source agent reliability layer that shows where, why, and how a run failed - and can replay from the most likely failure point.

2. Problem Tanımı ve Fırsat

2.1 Problem

Agent sistemlerinde HTTP 200 dönmesi veya tool çağrısının teknik olarak başarılı tamamlanması, görevin doğru yapıldığı anlamına gelmez. Agent yanlış aracı seçebilir, tool sonucunu ters yorumlayabilir, state'i hatalı güncelleyebilir, aynı döngüye girebilir, alt agent'a eksik bağlam aktarabilir veya zayıf bir kanıt nihai iddianın temeli yapabilir. Bu hatalar klasik log ve APM metriklerinde görünmeyebilir.

- Uzun-horizon akışlarda erken bir hata onlarca adım sonra görünür hâle gelir.
- Tek bir yanlış state güncellemesi tool seçimi, plan, hafıza ve nihai cevabı etkileyebilir.
- Trace sistemleri çoğu zaman ne olduğunu gösterir; kök neden adayını ve yayılım zincirini doğrulamaz.
- Bütün trace'i güçlü bir LLM'e tekrar tekrar göndermek maliyetli ve gizlilik açısından risklidir.
- Kullanıcı çoğu zaman tüm görevi yeniden başlatır; bu da zaman, token ve hesaplama kaybıdır.

2.2 Pazar Boşluğu

Langfuse agent graph, LangSmith Engine, Phoenix, AgentOps, Weave ve Opik gibi ürünler tracing, evaluation ve üretim izleme sunar. LangSmith Engine tekrarlanan sorunları ve kök nedenleri teşhis etmeye yönelmiştir; bu nedenle pazar tamamen boş değildir. Farklılaşma, yalnızca 'kök nedeni buluyoruz' ifadesiyle değil, aşağıdaki birleşimle kurulmalıdır:

- Terminal-native ve lokal çalışma
- Desteklenen runtime içinde sürekli fakat tokensız yapısal gözlem
- Failure onset localization ve downstream propagation graph
- Kanıt, güven ve belirsizlik etiketleri
- Checkpoint/session fork tabanlı doğrulanabilir replay
- Model bağımsız çalışma ve kullanıcıya ait provider anahtarları
- Açık benchmark, hata enjeksiyonu ve regresyon testleri

Rekabet açısından kritik uyarı

3B grafik bir farklılaştırıcı değil, sunum katmanıdır. Savunulabilir değer; yanlış pozitifleri düşük, kalibrasyonu ölçülmüş failure localization ve replay ile gösterilmiş kurtarma başarısıdır.

3. Proje Amacı, Hedefler ve Kapsam

3.1 Ana Amaç

Agent yürütmelerini güvenli ve düşük maliyetli biçimde kaydeden; teknik, yapısal ve semantik hata sinyallerini bir trace graph üzerinde birleştiren; en erken kritik hata adımını ve etki zincirini hesaplayan; kullanıcı onayıyla alternatif bir yürütme dalı oluşturan açık kaynak bir Runopsy adlı bağımsız Causal Failure Analysis Engine ürünü geliştirmek.

3.2 Ölçülebilir Hedefler

Hedef	Başarı ölçütü	Not
Tam trace yakalama	Enstrümente edilen LLM, tool, state, checkpoint ve sub-agent olaylarının şema doğrulaması	Eksik olaylar açıkça 'unknown' olarak işaretlenir
Tokensız temel analiz	Offline modda sıfır harici LLM çağırısı	Loop, exception, timeout, schema, exit code ve state invariant dedektörleri
Hata başlangıcı	Runopsy-Bench üzerinde top-1, top-3 ve mean step distance ölçümü	Hedef değerler pilot veri sonrası kilitlenir
Yayılım zinciri	Propagation edge precision/recall ve insan doğrulaması	Korelasyon ile nedensellik ayrılır
Replay	Başarısız koşulların kontrollü fork üzerinde kurtarma oranı	Aynı bütçe ve seed politikası ile karşılaştırma
Düşük ek yük	Deterministik modda gecikme, disk ve bellek overhead ölçümü	Geçici hedef: median duvar saati ek yükü <= %5
Güvenlik	Secret/PII maskeleyme testleri ve varsayılan local storage	Harici çağrı öncesi veri minimizasyonu

3.3 İlk Sürümde Kapsam Dışı

- Modelin gizli chain-of-thought içeriğine erişme veya bunu saklama
- Her semantik hatada kesin nedensellik garantisi
- Kullanıcı onayı olmadan otomatik ve geri döndürülemez self-healing
- Genel amaçlı enterprise observability platformunun tüm özelliklerini yeniden yapmak
- Kısa ve basit trace'lerde 3B arayüzü zorunlu kılmak
- Hermes çekirdeğini ilk sürümde fork etmek
- Ücretsiz OpenRouter modellerini üretim SLA'sı varmış gibi sunmak

4. Ürün Konumlandırması ve Kullanıcılar

4.1 Hedef Kullanıcılar

Persona	İhtiyaç	Runopsy değeri
Agent geliştiricisi	Hangi adımın görevi bozduğunu hızlı bulmak	Failure onset, state diff, evidence ve replay
AI araştırmacısı	Model ve agent stratejilerini aynı evaluator altında karşılaştırmak	Runopsy-Bench, seed tekrarı, trajectory metrikleri
Platform / MLOps ekibi	Üretim agent sorunlarını düşük maliyetle izlemek	OTel/OpenInference export, sampling, alerts
Güvenlik ekibi	Riskli tool çağrısını ve veri sızıntısını durdurmak	pre_tool_call policy ve evidence trail
Yerel model kullanıcıları	Kod ve trace'i dışarı göndermeden çalışmak	Ollama/llama.cpp + offline diagnosis

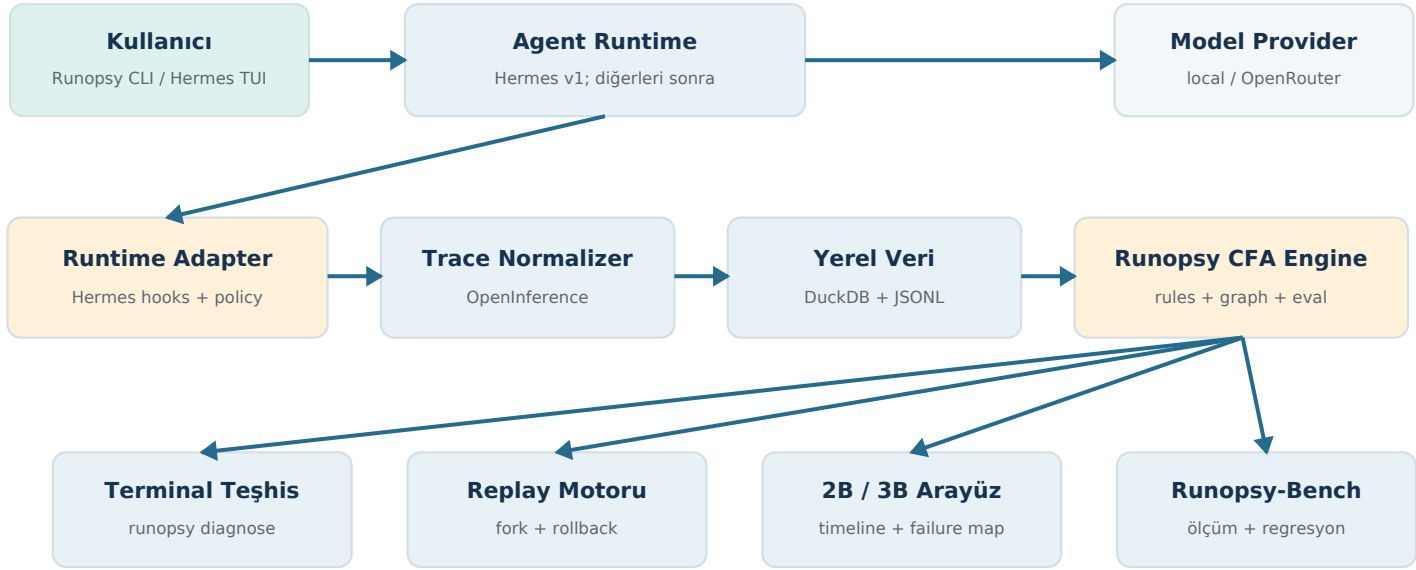
4.2 Temel Kullanım Senaryoları

- Kod düzeltme: Test çıkışı yanlış yorumlandığında ilk hatalı state güncellemesini bulma.
- Araştırma agent'ı: Kaynak seçimi sırasında zayıf kanıtın hangi iddiaları etkilediğini gösterme.
- Tool-use: Yanlış argüman veya başarısız exit code sonrasında planın devam etmesini yakalama.
- Multi-agent: Eksik handoff bağlamının alt agent çıktısına ve nihai sonuca yayılımını izleme.
- Maliyet kontrolü: Retry loop ve gereksiz model eskalasyonlarını belirleme.
- Güvenlik: Şüpheli terminal komutunu çalışmadan önce bloke etme ve gerekçe kaydetme.

4.3 Ürün Vaadi ve Rakiplerden Ayrışma

Kategori	Tipik ana değer	Bu projenin yaklaşımı
Genel agent CLI	Görevi yerine getirme	Görev + yerleşik failure analysis
Hafıza odaklı agent	Geçmişten öğrenme ve skill üretimi	Hafıza hatalarının state etkisini teşhis etme
Tracing platformu	Ne oldu, ne kadar sürdü?	Nerede bozuldu, neyi etkiledi, nasıl doğrulandı?
Evaluation platformu	Çıktıyı veya trajectory'yi puanlama	Evaluation sinyalini causal graph ve replay'e bağlama
3B workflow aracı	Mimarinin görsel keşfi	Görsel düğümlerin evidence ve replay ile eyleme dönüşmesi

5. Önerilen Mimari



Şekil 1. Runopsy ürün mimarisi: Hermes yalnızca ilk runtime adaptörüdür; çekirdek bağımsız çalışır.

5.1 Mimari İlkeler

- Bağımsız ürün sınırı: Runopsy kendi CLI, veri modeli ve analiz çekirdeğine sahip olur.
- Hermes'i fork etmeden başla: İlk adaptör plugin sistemi ve programmatic API kullanır.
- Framework bağımsız çekirdek: Runopsy CFA Engine yalnızca normalize edilmiş trace graph kabul eder.
- Local-first: Trace, state ve artifact varsayılan olarak kullanıcı cihazında saklanır.
- Deterministik-first: Teknik ve yapısal sinyaller model çağrısı olmadan değerlendirilir.
- Evidence-first: Her teşhis kaynak event, state diff ve evaluator sinyaliyle ilişkilendirilir.
- Belirsizliği görünür kıl: 'Gözlenen hata', 'kök neden adayı' ve 'doğrulanmış neden' ayrı statülerdir.
- İnsan kontrolü: Riskli müdahale ve replay açık kullanıcı onayı gerektirir.

5.2 Bileşenler

Bileşen	Sorumluluk	Girdi / Çıktı
Runopsy Hermes Adapter	Hook kaydı, runtime metadata, policy, snapshot ve slash command entegrasyonu	Hermes event -> Runopsy event
Trace Normalizer	Hermes olaylarını OpenInference uyumlu typed graph'a dönüştürür	Raw event -> TraceNode / TraceEdge
Local Collector	Append-only event alımı, sequence ve integrity kontrolü	Event stream -> JSONL / DuckDB
Detector Registry	Loop, exception, tool, state, handoff, evidence ve budget dedektörleri	Graph -> FailureSignal[]
Causal Ranker	Failure onset ve downstream impact adaylarını sıralar	Signals + graph -> Candidate[]
Semantic Evaluator	Yalnızca seçilen span'larda semantik çelişki ve claim-evidence analizi	Compressed evidence -> EvaluatorResult
Replay Orchestrator	Checkpoint, rollback, fork ve alternatif model/bağlam yürütmesi	Candidate + intervention -> ReplayRun
TUI / Web UI	Canlı durum, timeline, graph, evidence ve karşılaştırma	DiagnosisBundle -> kullanıcı görünümü
Runopsy-Bench	Etiketli trajectory, hata enjeksiyonu ve regresyon ölçümü	Runs + labels -> benchmark raporu

6. İlk Adaptör: Hermes Entegrasyon Tasarımı

Hermes plugin altyapısı LLM, tool, session ve sub-agent lifecycle olaylarını izlemeye izin verir. `pre_tool_call` eylemi engelleyebilir; `pre_llm_call` bağlam ekleyebilir. Diğer post hook'lar çoğunlukla observer olarak çalışır. Bu sınır, MVP'de gözlem ve kontrollü müdahale ayrımını belirler. (R2, R3)

Hermes hook / API	Runopsy kullanımı	Müdahale durumu
on_session_start	Run ID, repo, provider, model, policy ve başlangıç state'i	Gözlem
pre_llm_call	Turn snapshot; önceki teşhisten güvenli bağlam ekleme	Context injection
post_llm_call	Model yanıtı, tool planı, token, latency ve completion status	Gözlem
pre_tool_call	Argüman doğrulama, risk policy, loop ve destructive action kontrolü	Block / allow
post_tool_call	Tool output, exit code, artifact hash ve state delta	Gözlem
subagent_stop	Handoff input/output, eksik bağlam ve child outcome	Gözlem
on_session_end/finalize	Final outcome, diagnosis bundle, export ve benchmark sample	Gözlem
Checkpoint Manager	Dosya durumunu turn öncesine döndürme	Rollback
ACP / API session fork	Alternatif model veya talimatla ayrı dal oluşturma	Fork / replay

6.1 Plugin Paket Yapısı

```
runopsy-hermes-adapter/  
├── pyproject.toml  
├── runopsy_hermes/  
│   ├── plugin.py  
│   ├── hooks/  
│   │   ├── session.py  
│   │   ├── llm.py  
│   │   ├── tools.py  
│   │   └── subagents.py  
│   ├── commands/  
│   │   ├── diagnose.py  
│   │   ├── evidence.py  
│   │   ├── replay.py  
│   │   └── graph.py  
│   ├── mapping/  
│   │   └── hermes_to_runopsy.py  
│   ├── policy/  
│   │   └── defaults.yaml  
└── tests/
```

6.2 Plugin Sınırları ve Upstream Stratejisi

- Post hook'ların sonucu runtime davranışını doğrudan değiştiremez; teşhis sonucu sonraki turn veya replay akışında kullanılır.
- Hermes checkpoint'i dosya/turn odaklıdır; tam step-level replay için Runopsy ayrıca state snapshot tutar.
- Checkpoint rollback ile session fork birlikte kullanılırken dosya state'i ve mesaj geçmişinin aynı başlangıç noktasına ait olduğu doğrulanır.
- Hermes plugin API değişikliklerine karşı ince adapter ve contract testleri kullanılır.
- Gerekli hook eksikleri önce upstream katkı olarak önerilir; fork son seçenek olur.

Neden doğrudan fork önermiyoruz?

Fork kısa vadede kontrol sağlar; ancak hızlı değişen upstream kodla sürekli merge, güvenlik güncellemesi ve provider uyumluluğu yükü getirir. Plugin + companion engine, ürün değerini Hermes'in iç uygulamasından ayırır.

7. Trace ve Veri Modeli

7.1 Ana D ğ m T rleri

D�ğ�m	Temel alanlar	Ama
Run	run_id, task, repo, start/end, outcome	Bir agent y�r�tmesinin k�k�
Agent	agent_id, role, parent, model	Ana veya alt agent kimlięi
Turn	turn_id, input, output, order	Kullanıcı-agent etkileşim birimi
LLMCall	model, prompt_hash, tokens, latency, status	Model aęrısı
ToolCall	tool, args_hash, output, exit_code, retry	Dış eylem ve sonucu
StateSnapshot	schema_version, values, diff	Y�r�tme state'inin g�zlenen h�li
MemoryOp	read/write, key, source, freshness	Hafıza eriřimi
Claim	claim_id, text_hash, support_status	�retilen iddia
Evidence	source, excerpt_hash, quality, timestamp	İddiayı destekleyen veri
Artifact	path, type, hash, producer	Dosya, patch, tablo veya rapor
Checkpoint	checkpoint_id, repo_state, turn	Geri d�n�ř noktası
FailureSignal	type, severity, evidence, detector	Hata sinyali
Diagnosis	candidate, confidence, status, affected	K�k neden adayı ve yayılım
ReplayRun	parent_run, intervention, outcome	Karşı-olgusal y�r�tme

7.2 Ana Kenar T rleri

- PRECEDES: zaman sırası
- DEPENDS_ON: y r tme veya veri baęımlılıęı
- PRODUCED: d ğ m n  rettięi state/artifact/claim
- CONSUMED: kullanılan tool sonucu, memory veya evidence
- DERIVED_FROM: claim veya kararın kaynaęı
- CONTRADICTS: state, evidence veya iddia eliřkisi
- VALIDATES: test veya evaluator doęrulaması
- AFFECTS: olası downstream etki
- FORKED_FROM: replay ve orijinal kořu iliřkisi

7.3 Örnek Normalize Event

```
{
  "schema_version": "0.1",
  "event_id": "evt_0187",
  "run_id": "run_0042",
  "parent_id": "turn_0011",
  "kind": "tool_call",
  "sequence": 17,
  "timestamp": "2026-07-30T09:45:21+03:00",
  "agent_id": "main",
  "tool": {
    "name": "terminal",
    "arguments_hash": "sha256:...",
    "exit_code": 1,
    "duration_ms": 1824
  },
  "state_delta": {
    "tests_passed": {"before": false, "after": true}
  },
  "security": {
    "redacted": true,
    "contains_secret": false
  }
}
```

7.4 Saklama

Veri	MVP deposu	Politika
Structured event	DuckDB	Yerel, query edilebilir, schema version'lı
Raw event stream	Append-only JSONL	Debug ve yeniden işleme için
Artifact	Content-addressed lokal klasör	SHA-256, boyut limiti, secret scan
Graph cache	Parquet / DuckDB	Trace hash değişmedikçe yeniden kullan
Diagnosis	JSON	Evidence reference ve evaluator version içerir
Replay metadata	DuckDB	Parent run ve intervention zorunlu

8. Runopsy Causal Failure Analysis Engine



Kök neden çıktısı: gözlenen hata, kök neden adayı, güven, kanıt, etkilenen düğümler ve doğrulama durumu.

8.1 Analiz Katmanları

Katman	Yöntem	Token
L0 - Structural	Sequence, parent-child, missing span, schema, exit code, exception, timeout	Yok
L1 - Behavioral	Loop, retry storm, plan divergence, invariant ve state transition kuralları	Yok
L2 - Graph impact	Temporal precedence, dependency reachability, affected descendants ve centrality	Yok
L3 - Semantic	Tool sonucunu yanlış yorumlama, claim-evidence ve handoff completeness	Opsiyonel
L4 - Validation	Counterfactual replay, human label veya bağımsız evaluator	Replay'e bağlı

8.2 Aday Skorlama

Her adım i için örnek bileşik skor:

$$\text{failure_score}_i = \text{rule_severity} + \text{temporal_precedence} + \text{state_anomaly} + \text{downstream_impact} + \text{evaluator_support} - \text{uncertainty_penalty}$$

Ağırlıklar başlangıçta uzman kurallarıyla tanımlanır; Runopsy-Bench verisi oluştuğunda kalibre edilir. Skor olasılık olarak sunulacaksa Platt scaling, isotonic calibration veya uygun bir kalibrasyon yöntemiyle doğrulanmalıdır.

8.3 Teşhis Statüleri

Statü	Anlam	Kullanıcıya sunum
Observed failure	Doğrudan exception, exit code, invariant veya test hatası	Kesin gözlenen hata
Suspected onset	İlk güçlü anomali ve downstream ilişki	Kök neden adayı
Correlated cause	Zamansal ve yapısal ilişki var; müdahale yok	Doğrulanmamış ilişki
Replay-supported	Müdahale sonrası sonuç anlamlı biçimde düzeldi	Replay ile desteklenen neden
Human-verified	Uzman etiketi veya incelemesi var	İnsan doğrulanmalı
Unknown	Yeterli evidence yok	Bilinmiyor; tahmin yapılmaz

Bilimsel sınır

Bir adımın önce gelmesi ve sonraki hatalarla bağlantılı olması tek başına nedensellik kanıtı değildir. Ürün 'kesin kök neden' ifadesini yalnızca müdahale/replay veya güvenilir insan etiketi olduğunda kullanmalıdır.

9. Hata Taksonomisi

Kategori	Örnek	Tokensız sinyal	Semantik analiz
Goal / input	Eksik veya çelişkili amaç	Schema, missing field	Amaç tutarlılığı
Planning	Gereksiz veya eksik adım	Plan değişim sıklığı	Plan-goal uyumu
Retrieval	Alakasız/eski kaynak	Timestamp, source type	Relevance, authority
Tool selection	Yanlış aracın seçilmesi	Expected tool policy	Bağlama uygunluk
Tool arguments	Yanlış path/query/ID	Type/range validation	Niyet-argüman uyumu
Tool execution	Exit code, exception, timeout	Doğrudan sinyal	Genellikle gerekmez
State	Sonuçla çelişen state update	Invariant, typed diff	Anlam uyumsuzluğu
Memory	Eski veya yanlış hafıza	Freshness, scope	Hatırlanan bilginin geçerliliği
Handoff	Alt agent'a eksik bağlam	Required field check	Completeness
Reasoning	Desteksiz çıkarım	Claim graph gap	Entailment/judge
Validation	Test hatasını kaçırma	Result/status mismatch	Test yorumlama
Control flow	Loop/retry storm	Cycle/repetition	Gerekmez
Budget	Aşırı token veya latency	Threshold/rate	Gerekmez
Safety	Riskli komut/veri sızıntısı	Policy/regex/secret scan	Bağlamsal risk
Outcome	Görev tamamlandı denip tamamlanmaması	Acceptance check	Task completion

10. Replay ve Kurtarma Tasarımı

10.1 Replay Seviyeleri

Seviye	Mekanizma	MVP durumu	Risk
R0 - Explain only	Yürütmeyi değiştirmeden teşhis	MVP	Düşük
R1 - Turn rollback	Hermes checkpoint ile dosyaları turn öncesine al	MVP	Orta
R2 - Session fork	Mesaj geçmişini ayrı session'a kopyala	MVP	Orta
R3 - Guided replay	Fork + model/prompt/tool policy müdahalesi	MVP sonrası	Orta/Yüksek
R4 - Step replay	Her tool/state adımından tam snapshot	İleri sürüm	Yüksek
R5 - Automated recovery	Policy sınırlarında otomatik müdahale	Araştırma	Çok yüksek

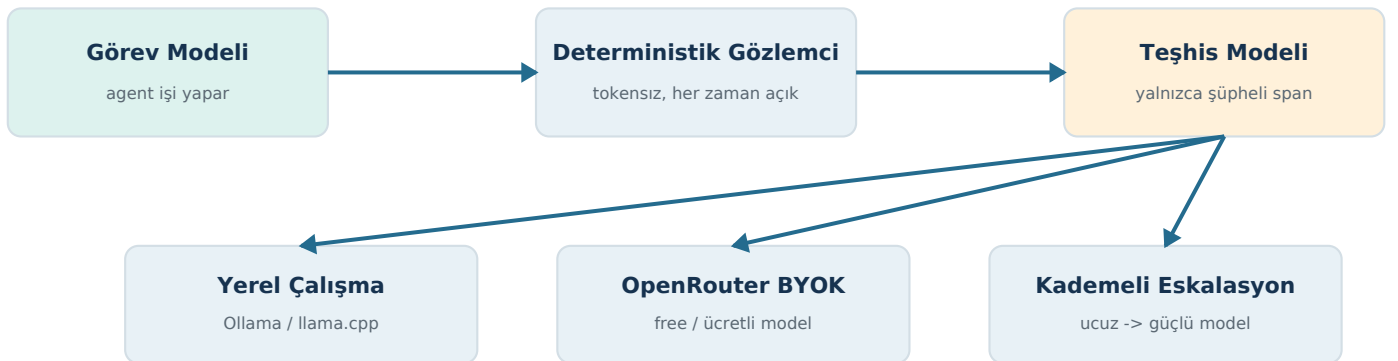
10.2 Replay Güvenlik Kontrolleri

- Replay varsayılan olarak fork/worktree veya güvenli sandbox içinde çalışır.
- Dosya checkpoint'i ile session başlangıç state'i aynı sequence numarasına bağlanır.
- Harici yan etkili tool'lar dry-run, mock veya kullanıcı onayı olmadan tekrar çalıştırılmaz.
- E-posta, ödeme, silme, yayınlama ve uzak sistem değişiklikleri otomatik replay dışında tutulur.
- Original run değişmez; replay ayrı run_id ve parent_run_id ile saklanır.
- Model, prompt, seed, tool sürümü ve environment farkları intervention kaydına yazılır.

10.3 Counterfactual Doğrulama

Aday adım değiştirilip downstream sonuç düzeliyorsa teşhis desteklenir; ancak aynı anda birden fazla değişken değiştirilmemelidir. Model, prompt ve tool çıktısı ayrı intervention testleri olarak yürütülmelidir. Non-determinism için birden fazla seed ve matched budget kullanılır.

11. Model ve Provider Stratejisi



11.1 Kullanıcı Kurulumu

Kullanıcı yerel modeli seçtiğinde ürün donanımı tespit eder ve uyumlu runner/model önerir. Model habersiz indirilmez. Boyut, lisans, context ihtiyacı, disk alanı ve beklenen hız gösterildikten sonra açık onay alınır. Hermes'in güncel çok adımlı kullanımında en az 64K context beklentisi, yerel model seçiminde temel kısıttır. (R4)

Mod	Görev modeli	Teşhis modeli	Maliyet / gizlilik
Tam lokal	Ollama veya llama.cpp	Ayrı lokal model veya rules-only	API maliyeti yok; donanım maliyeti var
Hibrit	OpenRouter BYOK	Lokal model	Görev token'ı var; trace dışarı çıkmaz
Hibrit ters	Lokal model	OpenRouter yalnız şüpheli span	Düşük token; seçilmiş evidence dışarı çıkar
Tam OpenRouter	Kullanıcı modeli	Farklı diagnostic model	En kolay kurulum; veri politikası önemlidir
Demo	openrouter/free	Rules-only veya free	Ücretsiz; rate limit ve model sürekliliği garantisiz

11.2 Actor ve Diagnostician Ayrımı

- Görevi yapan model ile teşhis modelinin rolü ve bütçesi ayrı tanımlanır.
- Aynı modelin kendisini değerlendirmesi tek başına güvenilir kabul edilmez.
- Önce rule-only tarama, sonra küçük/ucuz model, yalnızca düşük güven durumunda güçlü model kullanılır.
- Her evaluator sonucu model kimliği, prompt sürümü ve evidence hash ile saklanır.

12. Token ve Maliyet Kontrolü

Aşama	Token tüketimi	Kontrol
Trace yakalama	Yok	Lokal event stream
Graph oluşturma	Yok	NetworkX / DuckDB
Deterministik dedektör	Yok	Her trace için açık
Embedding	Seçime bağlı	Lokal embedding veya kapalı
Semantic evaluator	Var	Yalnız şüpheli span ve sıkıştırılmış evidence
Diagnosis explanation	Var	Kısa structured output ve cache
Counterfactual replay	Yüksek olabilir	Kullanıcı onayı, run budget ve early stop

12.1 Bütçe Modları

```
analysis:
  mode: deterministic      # deterministic | hybrid | deep
  llm_on: suspicious      # never | suspicious | always
  max_diagnostic_calls: 2
  max_diagnostic_input_tokens: 6000
  max_replay_runs: 1
  max_cost_usd: 0.10
  cache_by_trace_hash: true
```

12.2 Veri Minimizasyonu

- Tam trace yerine hedef, şüpheli adım öncesi/sonrası state diff, tool sonucu ve ilgili evidence gönderilir.
- Dosya içeriği yerine mümkün olduğunda hash, path sınıfı ve güvenli özet kullanılır.
- Secret scanner sonrası maskelenemeyen trace harici modele gönderilmez.
- Free provider kullanımı ayrıca veri saklama ve provider policy kontrolüne tabidir.

13. Teknoloji Yığını ve Kütüphaneler

13.1 Backend ve CLI

Teknoloji	Rol	Seçim gerekçesi
Python 3.12	Ana geliştirme dili	Hermes ve AI/eval ekosistemiyle uyum
Typer	Companion CLI	Typed command ve otomatik help
Textual + Rich	Terminal TUI	Canlı tree, table, log ve keyboard UX
Pydantic v2	Typed trace/config	Şema doğrulama ve güvenli serialization
OpenTelemetry SDK	Telemetry taşıma	Vendor-neutral standart
OpenInference semconv	AI span anlamı	LLM/tool/agent semantic conventions
DuckDB	Yerel analitik depo	Tek dosya, SQL, Parquet entegrasyonu
NetworkX	MVP graph analizi	Hızlı prototip, reachability ve centrality
FastAPI + Uvicorn	Lokal API	Typed API ve async streaming
httpx	Provider/API istemcisi	Async, timeout ve test edilebilir transport
orjson	Event serialization	Hızlı ve deterministik JSON
structlog	Uygulama logları	Structured logging ve redaction pipeline
psutil	Donanım keşfi	CPU/RAM/process ve kurulum kontrolü

13.2 Model ve Evaluation

Araç	Rol	Zorunluluk
Hermes Agent	Görev runtime ve plugin host	MVP için zorunlu
Ollama	Kolay lokal model yönetimi	Opsiyonel
llama.cpp	Düşük seviyeli lokal inference	Opsiyonel
OpenRouter	Model-bağımsız BYOK gateway	Opsiyonel
Inspect AI	Runopsy-Bench agent eval ve sandbox	Benchmark için önerilen
Arize Phoenix	Açık kaynak trace inceleme ve karşılaştırma	Opsiyonel entegrasyon

13.3 Web ve Görselleştirme

Teknoloji	Rol	Not
React + TypeScript	Lokal web arayüzü	Typed UI bileşenleri
Vite	Build ve dev server	Hızlı lokal geliştirme
React Flow / XYFlow	2B DAG ve trace görünümü	Varsayılan debugging görünümü
Three.js + React Three Fiber	3B failure map	Multi-agent ve uzun-horizon için
TanStack Query	API state ve cache	Trace/diff sorguları
Zustand	Lokal UI state	Seçim, filtre, kamera ve replay state

13.4 Kalite, Paketleme ve Dağıtım

Araç	Kullanım
pytest + pytest-asyncio	Unit, integration ve async event testleri
Hypothesis	Trace graph ve schema property-based testleri
coverage.py	Test kapsamı
Ruff	Lint ve format
mypy	Statik tip kontrolü
Bandit + Semgrep Community	Python güvenlik kontrolleri
uv + hatchling	Bağımlılık, build ve reproducible environment
pipx / uv tool	CLI kurulumu
Docker	Opsiyonel sandbox ve demo deployment
GitHub Actions	Public repo için CI, release ve dependency scan

Lisans politikası

Tüm doğrudan ve geçişli bağımlılıklar SBOM ve lisans taramasından geçirilmelidir. Hermes çekirdeğinin MIT olması, her opsiyonel skill veya üçüncü taraf MCP'nin ticari kullanıma uygun olduğu anlamına gelmez.

14. Companion CLI ve Kullanıcı Deneyimi

```
# Kurulum ve sağlık kontrolü
runopsy setup
runopsy doctor

# Hermes adaptör durumunu doğrula
runopsy adapter hermes status

# Son koşuyu tokensız analiz et
runopsy diagnose latest --mode deterministic

# Yalnız şüpheli span için semantik analiz
runopsy diagnose latest --mode hybrid --budget-usd 0.10

# Evidence ve yayılım zinciri
runopsy evidence latest --step 17
runopsy graph latest --view causal

# Kontrollü replay
runopsy replay latest --from-step 17 --model local:qwen --dry-run

# Lokal web arayüzü
runopsy ui

# Paylaşılabilir, redacted rapor
runopsy export latest --format html --redact
```

14.1 Hermes İçindeki Runopsy Slash Komutları

Komut	İşlev
/runopsy status	Aktif policy, trace ve bütçe durumu
/runopsy diagnose	Son veya seçili koşuyu analiz eder
/runopsy evidence 17	17. adımın kanıt, state diff ve descendants görünümü
/runopsy graph	Lokal web graph görünümünü açar
/runopsy replay 17	Kullanıcı onaylı fork/rollback planı hazırlar
/runopsy mode offline	Harici LLM çağrılarını tamamen kapatır
/runopsy budget 0.10	Teşhis ve replay harcama tavanı

14.2 Lokal Model Onboarding

- OS, mimari, CPU, RAM, GPU/VRAM ve disk tespit edilir.
- Uyumlu runner ve context ihtiyacını karşılayan model önerilir.
- Model boyutu, kaynak, lisans, quantization ve tahmini bellek ihtiyacı gösterilir.
- Açık onay sonrası indirme yapılır; checksum ve health inference doğrulanır.
- Uygun donanım yoksa OpenRouter BYOK veya rules-only mod önerilir.

15. 2B ve 3B Görselleştirme Tasarımı

15.1 Varsayılan 2B Görünüm

- Timeline/waterfall: zaman, latency, token, retries
- Execution DAG: agent, tool, state ve handoff ilişkileri
- State diff paneli: önceki/sonraki değerler
- Evidence paneli: kaynak event ve evaluator sinyali
- Failure ribbon: gözlenen hata, aday onset ve etkilenen descendants

15.2 Opsiyonel 3B Failure Map

Eksen / kodlama	Anlam
X eksen	Adım veya zaman
Y eksen	Agent, tool veya paralel dal
Z eksen	Evidence integrity, validation strength veya state health
Renk	Sağlıklı, kök hata aday, etkilenen, doğrulanmış
Kenar	Dependency, derived_from, handoff veya affects
Node boyutu	Downstream impact veya centrality
Animasyon	Trace replay; başlangıç animasyonu değil, kullanıcı kontrollü adım geçişi

UX ilkesi

3B görünüm küçük trace için varsayılan olmamalıdır. Occlusion ve bilişsel yük ölçülmeli; kullanıcı her 3B düğümü 2B timeline ve ham evidence ile doğrulayabilmelidir.

16. Güvenlik, Gizlilik ve Yönetişim

Risk	Kontrol
API key sızıntısı	Keyring/env; trace öncesi secret scanner; hiçbir key loglanmaz
Kod/veri dışarı çıkması	Default local; semantic payload minimizasyonu; açık provider onayı
Riskli tool replay	Dry-run, policy gate, human approval, sandbox/worktree
Prompt injection	Tool output trust label; untrusted evidence ayrımı; policy
Yanlış kök neden	Belirsizlik, evidence, top-k aday ve replay doğrulaması
Trace manipülasyonu	Sequence, hash chain veya append-only integrity metadata
PII	Configurable masking ve retention; export sırasında redaction
Supply chain	Lockfile, SBOM, signed release, dependency scan
Plugin yetkileri	Minimum yetki; hook kapsamı açık; opt-in enable
3B yanlış güven	Her görsel node ham event ve evaluator sürümüne linkli

16.1 Varsayılan Güvenli Mod

```
privacy:
  storage: local
  send_raw_trace_to_provider: false
  redact_secrets: true
  redact_pii: true
  retain_raw_days: 7

replay:
  require_confirmation: true
  sandbox: worktree
  external_side_effects: block

diagnosis:
  causal_language: calibrated
  allow_definitive_root_cause_without_replay: false
```

17. Runopsy-Bench: Bilimsel Doğrulama Planı

Runopsy-Bench, ürünün asıl savunulabilir çıktısıdır. Inspect AI; dataset, agent, tool, scorer ve sandbox bileşenleri nedeniyle benchmark harness'i olarak kullanılabilir. Inspect AI açık kaynak MIT lisanslıdır ve agent evaluation desteği sunar. (R12)

17.1 Veri Katmanları

Katman	İçerik	Etiket
Synthetic unit traces	Tek hata türü ve deterministik graph	Kesin onset/edge
Fault-injected Hermes runs	Kontrollü tool/state/handoff hataları	Injection step
Public agent trajectories	Kod düzeltme ve tool-use görevleri	Uzman inceleme
Real opt-in failures	Redacted kullanıcı koşulları	Human verified
Replay pairs	Original ve intervention run	Recovery outcome

17.2 Metrikler

- Failure onset: top-1 accuracy, top-3 recall, mean absolute step distance
- Root-cause classification: macro-F1 ve per-category precision/recall
- Propagation: affected-edge precision/recall ve path coverage
- Calibration: Brier score, expected calibration error ve reliability plot
- Recovery: task success delta ve replay success rate
- Efficiency: additional tokens, wall time, storage ve LLM calls
- Safety: false-block rate ve missed destructive action rate
- Human value: time-to-diagnosis ve reviewer agreement

17.3 Karşılaştırılacak Baseline'lar

Baseline	Amaç
No diagnosis	Yalnız trace ve manuel inceleme
Rule-only	LLM olmadan yapısal analiz
Full-trace LLM	Tüm trajectory tek modele gönderilir
Candidate-first hybrid	Önerilen deterministik + seçili semantic yaklaşım
Replay-supported	Hybrid + counterfactual validation
Human expert	Üst sınır ve inter-rater agreement

Araştırma özgünlüğü

Literatürde CodeTracer gibi failure onset localization ve reflective replay çalışmaları bulunmaktadır. Özgün katkı; Hermes-native local-first ürünleştirme, kanıt grafi, token-budgeted hybrid diagnosis, calibrated causal language ve multi-agent propagation doğrulaması üzerinden kurulmalıdır. (R13)

18. Test Stratejisi ve Kalite Kapıları

Test katmanı	Örnek kalite kapısı
Unit	Her detector için pozitif/negatif fixture ve edge case
Schema	Version migration, invalid event ve unknown field testleri
Property-based	Acyclic/cyclic graph, missing spans, event ordering
Integration	Hermes hook -> collector -> diagnosis uçtan uca
Replay	Checkpoint/session eşleşmesi ve external side-effect block
Security	Secret/PII redaction, path traversal, unsafe command policy
Performance	10K/100K event ingest, graph build ve query benchmark
UI	Node-evidence eşleşmesi, keyboard accessibility, large trace
Compatibility	Desteklenen Hermes sürümleri için contract matrix
Release	SBOM, license scan, signed artifact, changelog

18.1 MVP Kabul Kriterleri

- Hermes'ten LLM/tool/session/sub-agent event'leri kaydedilebilir.
- Offline modda hiçbir harici model çağrısı yapılmaz.
- En az sekiz deterministik failure detector uçtan uca çalışır.
- Diagnosis JSON her aday için evidence, confidence ve affected nodes içerir.
- Turn-level rollback ve session fork için dry-run planı üretilebilir.
- Riskli external side-effect replay varsayılan olarak engellenir.
- 2B timeline ve causal graph aynı trace node kimliklerini kullanır.
- Synthetic benchmark raporu tekrarlanabilir şekilde üretilebilir.
- Dokümantasyon, örnek proje ve bir komutla lokal demo sağlanır.

19. Repository ve API Tasarımı

19.1 Monorepo Önerisi

```
causal-failure-engine/
├── packages/
│   ├── runopsy-core/           # graph, schema, detector, ranking
│   ├── adapter-hermes/         # Hermes plugin ve mapping
│   ├── runopsy-collector/      # local ingest ve persistence
│   ├── runopsy-replay/        # checkpoint/fork orchestration
│   ├── runopsy-cli/           # Typer + Textual
│   ├── runopsy-server/        # FastAPI
│   ├── runopsy-ui/            # React 2B/3B
│   └── runopsy-inspect/       # Inspect AI benchmark adapter
├── benchmarks/
│   ├── synthetic/
│   ├── fault_injection/
│   └── labeled_runs/
├── examples/
│   ├── coding_failure/
│   ├── research_failure/
│   └── multi_agent_handoff/
├── docs/
├── scripts/
├── tests/
├── pyproject.toml
├── uv.lock
└── LICENSE
```

19.2 Lokal API

Endpoint	İşlev
POST /v1/events	Tek veya batch normalize event alımı
GET /v1/runs	Run listesi ve filtre
GET /v1/runs/{id}	Run özeti
GET /v1/runs/{id}/graph	Node/edge graph
POST /v1/runs/{id}/diagnose	Analiz başlatma
GET /v1/diagnoses/{id}	Teşhis, evidence ve confidence
POST /v1/runs/{id}/replay/plan	Dry-run intervention planı
POST /v1/runs/{id}/replay	Onaylı replay
GET /v1/runs/{id}/stream	SSE canlı event
POST /v1/export	Redacted HTML/JSON export

20. Yol Haritası ve Karar Kapıları

Faz	Çıktı	Karar kapısı
F0 - Discovery	Hermes hook spike, lisans/SBOM, 20 örnek failure	Gerekli event'lerin $\geq$ %90'ı plugin ile yakalanıyor mu?
F1 - Trace foundation	Schema, collector, DuckDB, event viewer	Event ordering ve data integrity testleri geçiyor mu?
F2 - Deterministic engine	8+ detector, candidate ranking, diagnosis JSON	Synthetic localization hedefi karşılanıyor mu?
F3 - Hermes UX	Slash commands, TUI, 2B graph, budget modes	Manuel debug süresinde anlamlı azalma var mı?
F4 - Replay	Rollback, fork, dry-run ve guided replay	Yan etkisiz görevlerde recovery artıyor mu?
F5 - Runopsy-Bench	Fault injection, Inspect adapter, benchmark raporu	Hybrid model rule-only ve LLM-only baseline'ı geçiyor mu?
F6 - Public beta	PyPI/uv tool, docs, demo, signed release	Kullanıcı kurulumu ve crash telemetry olmadan debug edilebilir mi?
F7 - Framework expansion	OpenInference generic ve OpenCode adapter	Hermes dışı talep ürün odağını doğruluyor mu?

20.1 Öncelik Sırası

- Önce event capture ve schema doğruluğu
- Sonra deterministik failure engine
- Ardından evidence paneli ve 2B debugging UX
- Daha sonra replay ve semantic evaluator
- En son 3B görünüm ve cloud özellikleri

21. Risk Kaydı

Risk	Etki	Olasılık	Azaltım
Hermes API/hook değişimi	Yüksek	Orta	İnce adapter, version matrix, contract test
Yanlış causal claim	Çok yüksek	Orta	Calibrated language, evidence, replay/human validation
Local model yetersizliği	Orta	Yüksek	Hardware check, rules-only, BYOK fallback
Token maliyeti	Orta	Orta	Candidate-first, cache, budgets, sampling
Trace veri sızıntısı	Çok yüksek	Orta	Local default, masking, payload minimization
Replay yan etkisi	Çok yüksek	Düşük/Orta	Dry-run, sandbox, approvals, external block
3B kullanılabilirlik	Orta	Orta	2B default, usability test, evidence linking
Depolama büyümesi	Orta	Yüksek	Retention, compression, artifact dedupe
Lisans uyumsuzluğu	Yüksek	Orta	SBOM, allowlist, legal review
Rakiplerin hızla benzemesi	Yüksek	Yüksek	Benchmark, dataset, research report, plugin community
Plugin olarak algılanma	Orta	Orta	Bağımsız engine/CLI ve OpenInference API

22. Açık Kaynak ve Piyasaya Çıkış Stratejisi

22.1 Lisans ve Ürün Modeli

Katman	Öneri
Runopsy Core	Apache-2.0 önerisi; patent grant ve kurumsal kullanım açısından uygun
Hermes plugin	Apache-2.0 veya MIT; Hermes MIT bildirimleri korunur
Benchmark/data	Veri kaynağına göre ayrı lisans; kullanıcı trace'i açık rıza olmadan yayımlanmaz
Lokal ürün	Tam işlevli açık kaynak temel deneyim
Gelecek cloud	Opsiyonel hosted storage, team collaboration, alerts ve enterprise controls

22.2 İlk Dağıtım Kanalları

- GitHub: kaynak kod, issue templates, roadmap ve benchmark
- PyPI / uv tool: CLI ve Hermes plugin kurulumu
- Hermes plugin ekosistemi: ilk hedef kullanıcı topluluğu
- Hugging Face: Runopsy-Bench dataset ve demo
- Teknik rapor: failure localization ve replay sonuçları
- OpenRouter: BYOK entegrasyonu ve uygun olduğunda app attribution

22.3 Görünürlük İçin Kanıt Paketi

- 60 saniyelik terminal demo: hata tespiti -> evidence -> replay
- Üç gerçek failure case study
- Rule-only, LLM-only ve hybrid karşılaştırması
- Token/maliyet ve recovery tablosu
- Tek komutla tekrarlanabilir benchmark
- Public roadmap ve contribution guide

23. Uygulama Kararları

Konu	Karar
Hermes kullanımı	Evet; ilk runtime ve dağıtım yüzeyi
Hermes fork	Hayır; plugin sınırı kanıtlanana kadar
Bağımsız engine	Evet; normalize trace graph üstünde
Ücretli bağımlılık	Zorunlu değil
Local model	Kullanıcı onaylı, donanım ve lisans kontrollü
OpenRouter	BYOK; ücretsiz router yalnız demo/düşük hacim
Her trace için LLM	Hayır
3B arayüz	Opsiyonel; 2B varsayılan
Kesin kök neden ifadesi	Replay veya insan doğrulaması olmadan kullanılmaz
Ana ürün metrikleri	Localization, calibration, recovery, overhead ve false-block

Nihai öneri

MVP'yi bağımsız Runopsy CLI olarak çıkarın; Hermes adaptörünü ilk gün destekleyin. Çekirdeği OpenInference uyumlu ve framework bağımsız tasarlayın. Böylece Hermes kullanıcılarıyla hızlı doğrulama yapılırken ürün kimliği tek bir runtime'a bağlanmaz.

24. İlk Sprint İçin Somut İş Listesi

Sıra	İş	Çıktı
1	Hermes plugin skeleton ve hook spike	LLM/tool/session events JSONL
2	Trace schema v0.1	Pydantic modelleri ve JSON Schema
3	DuckDB collector	Run/event/query tabloları
4	Deterministik detector registry	Exception, exit code, loop, timeout, state mismatch
5	Diagnosis bundle	candidate, evidence, confidence, affected nodes
6	CLI diagnose/evidence	İlk kullanılabilir terminal deneyimi
7	Synthetic failures	20 fixture ve ground-truth onset
8	2B HTML graph	Timeline + causal edges
9	Checkpoint/fork dry-run	Replay planı; gerçek yan etki yok
10	Benchmark raporu	Rule-only ilk baseline

25. Kaynaklar

- R1. Hermes Agent Documentation - ürün, CLI ve açık kaynak agent genel bakış.
<https://hermes-agent.nousresearch.com/docs/>
- R2. Hermes Agent Event Hooks - pre/post LLM ve tool hook davranışları.
<https://hermes-agent.nousresearch.com/docs/user-guide/features/hooks>
- R3. Hermes Agent Plugin Development - plugin, hook, slash command ve extension yüzeyleri.
<https://hermes-agent.nousresearch.com/docs/developer-guide/plugins>
- R4. Hermes Agent Providers ve Quickstart - Ollama/yerel modeller ve context gereksinimi.
<https://hermes-agent.nousresearch.com/docs/integrations/providers>
- R5. Hermes Agent Checkpoints and Rollback - shadow git checkpoint mekanizması.
<https://hermes-agent.nousresearch.com/docs/user-guide/checkpoints-and-rollback>
- R6. Hermes Agent Programmatic Integration - ACP, session fork, cancel ve tool events.
<https://hermes-agent.nousresearch.com/docs/developer-guide/programmatic-integration>
- R7. Hermes Agent MIT License.
<https://github.com/NousResearch/hermes-agent/blob/main/LICENSE>
- R8. OpenTelemetry Documentation - vendor-neutral traces, metrics ve logs.
<https://opentelemetry.io/docs/>
- R9. OpenInference - OpenTelemetry için AI observability semantic conventions.
<https://github.com/Arize-ai/openinference>
- R10. Arize Phoenix - açık kaynak tracing, evaluation ve self-host yaklaşımı.
<https://arize.com/docs/phoenix>
- R11. OpenRouter Free Models Router - ücretsiz kullanım ve rate limit/availability sınırlamaları.
<https://openrouter.ai/docs/guides/routing/routers/free-router>
- R12. Inspect AI - açık kaynak agent evaluation, tools, scorers ve sandbox framework'ü.
<https://inspect.aisi.org.uk/>
- R13. Li, H. ve diğerleri (2026). CodeTracer: Towards Traceable Agent States.
<https://arxiv.org/abs/2604.11641>
- R14. LangSmith Engine - recurring issue detection, root-cause diagnosis ve fix/eval loop.
<https://docs.langchain.com/langsmith/engine-overview>
- R15. Langfuse Agent Graphs - aggregated ve expanded agent execution graph görünümü.
<https://langfuse.com/docs/observability/features/agent-graphs>
- R16. Llama.cpp - yerel ve bulut ortamlarında düşük seviyeli LLM inference.
<https://github.com/ggml-org/llama.cpp>

Kaynak kontrol tarihi: 30 Temmuz 2026. Kütüphane sürümleri, lisanslar, model erişimi ve platform özellikleri yayımdan önce yeniden doğrulanmalıdır.